Reject-Before-Propagate: Build-Time Citation Gates for Agentic Literature-Synthesis Pipelines

Harish Vidyarth N[a,*], A.L. Reshma[a], R. Divya[a]

[a]Department of Computer Science and Engineering (Cyber Security),
Rajalakshmi Engineering College, Thandalam, Chennai 602105, India

Abstract

Large language model (LLM) pipelines for automated literature review increasingly chain several generative stages – claim extraction, contradiction detection, evidence synthesis – whose outputs are consumed by downstream stages and, ultimately, by people making real decisions. In current systems, citation grounding is treated as a *scored* property: a generated statement is checked against retrieved evidence and assigned a confidence value, flagged, or routed for human review, but is not structurally prevented from reaching the final output when that check fails. We describe **Verify-or-Reject Grounding (VoRG)**, an architectural pattern that instead treats citation grounding as a *build-time invariant*: each pipeline stage either produces output whose type excludes unverifiable content – because the stage is deterministic over already-verified inputs, or because an explicit check discards non-conforming output before it can propagate – or it raises a build error. We give a formal statement of the pattern, a linear-time certification algorithm for its narrative-verification instance, and a reference implementation that applies it to three subsystems of an eight-stage research-assistant pipeline (evidence-grounded synthesis, contradiction detection, and journal-metadata reconciliation). We evaluate on one gold-annotated topic(s) with three repeated runs per configuration, comparing the gated pipeline against a matched *scored-grounding* ablation that keeps the same stage decomposition but replaces each rejection with a confidence score, and against a naive single-prompt baseline. The most robust finding is diagnostic rather than a headline metric: the pipeline stages protected by a verify-or-reject gate produce no output contradicted by the reviewed corpus, while research-gap detection – the one major stage with no such gate – produces a confirmed false claim directly contradicted by a paper already in the corpus. This gap-to-failure correspondence, between which stages are architecturally protected and where empirical failures occur, is reported as a concrete design finding and motivates extending the same discipline to every generative stage of an agentic scientific-NLP pipeline.

Keywords: verify-or-reject grounding; literature synthesis; large language models; citation grounding; hallucination mitigation; agentic NLP; scientific NLP

*Corresponding author: harishvidyarth.n.2024.csecs@rajalakshmi.edu.in

1 Introduction

Large language models are increasingly used to accelerate literature review by extracting claims from papers, identifying disagreements between studies, and drafting synthesis narratives. A well-documented failure mode of this automation is *hallucination*, in which a generated statement appears well supported but does not correspond to any statement in the source [1–3]. A closely related failure is citation-level: even when the surrounding text is accurate, the cited source may not actually say what the model claims, or may not exist at all [4]. Existing mitigation approaches commonly rely on post-hoc verification, in which a claim or generated passage is checked against retrieved evidence and assigned a score, flagged, or routed for human review [5]. Because validation in this scheme is a signal appended to the output rather than a gatekeeper that controls it, an unverified or contradictory statement can still reach the user if the score is not acted on.

This paper describes an alternative design principle, **Verify-or-Reject Grounding (VoRG)**. The distinction matters because synthesis output from pipelines of this kind is often consumed downstream by people making real decisions, such as a systematic-review team drafting a submission [5]; a hallucinated citation that merely lowers a confidence score, rather than being blocked outright, carries a correspondingly higher cost in that setting. Section 4 formalizes VoRG's rejection semantics and gives a linear-time certification algorithm for the narrative-verification case. Section 5 reports a reproducible evaluation whose most informative result is a correspondence between architectural coverage and failure location rather than an effect size.

Our contributions are:

1. A formal statement of Verify-or-Reject Grounding as a reusable architectural pattern (§4), distinguished from scored or soft grounding by its build-time rejection semantics, together with VoRG-Certify, an $O(n+M)$ narrative-certification algorithm that removes the $O(n \cdot m)$ term implicit in a naive per-sentence evidence scan.

2. A reference implementation applying the pattern to three subsystems of an eight-stage research-assistant pipeline – evidence-grounded synthesis, contradiction detection, and journal-metadata reconciliation (§3) – released with its evaluation harness and gold-annotation files.

3. A reproducible evaluation (§5) that compares the gated pipeline against a *matched scored-grounding ablation* – the same stage decomposition with every rejection replaced by a confidence score – and a naive single-prompt baseline, and reports a **gap-to-failure correspondence**: the verify-or-reject-gated stages produce no output contradicted by the reviewed corpus, whereas the single ungated stage produces a false claim that a corpus paper directly contradicts.

The remainder of the paper is organized as follows. Section 2 reviews related work on automated literature review, contradiction detection, citation grounding, and bibliometric reconciliation. Section 3 describes the system architecture, Section 4 presents the verify-or-reject pattern and its certification algorithm, and Section 5 reports the evaluation. Discussion, threats to validity, and reproducibility are presented in Sections 6–8, followed by the conclusion.

2 Related Work

AI-assisted systematic review. Li et al. [5] develop a GPT-4-driven, five-module system (literature search, PICOs criteria setup, abstract screening, data extraction, and reporting) for systematic literature reviews conducted as part of health technology assessment (HTA) submissions, with a human-in-the-loop design that permits real-time criteria refinement. Evaluated on 3,713 relapsed/refractory multiple myeloma abstracts and 1,720 advanced-melanoma abstracts, the system reports 90% average sensitivity (82 to 97%) and an F1 score of 82 (69 to 90) for abstract screening, and an F1 score of 93 (precision 91%, recall 94%) for data extraction. These figures show the system performs well within its evaluated domain, but its grounding safeguard is a human review step, not a structural constraint on what the model can emit: a low-confidence screening or extraction decision is still surfaced to the reviewer rather than withheld. This is the closest published system to the reference implementation described here and is treated as this paper's base comparison: where Li et al. [5] score output and route it for human review, our pipeline rejects unverifiable output automatically, without a human in the loop at generation time. Two more recent LLM-assisted review tools broaden this landscape without changing the underlying pattern: Haryanto's LLAssist [6] automates relevance screening and information extraction with an LLM scoring each paper's fit to a research question, and Scherbakov et al. [7] systematically review 172 LLM-automated-review studies and find that the large majority automate a single stage (most often search or data extraction) with a confidence score or human check, rather than a chained, structurally-gated pipeline.

Automation of systematic review more broadly. van Dinter et al. [8] systematically review the SLR-automation literature and find that most automation efforts target individual review stages (search, screening, or data extraction) in isolation, with few systems chaining verifiable stages end-to-end into a single pipeline with consistent grounding guarantees, a gap the architecture in §3 is designed to close. The broader retrieval-augmented and agentic-LLM literature offers relevant building blocks without addressing this gap directly: retrieval-augmented generation [9] grounds generation in retrieved passages but does not reject ungrounded output, ReAct-style tool-using agents [10] interleave reasoning and external actions without a citation-level accept/reject rule, and Self-RAG [11] trains a model to critique its own retrieval and generation via learned reflection tokens, which is closer in spirit to VoRG but remains a soft, learned signal rather than a hard, external verification gate.

Contradiction detection. Alamri and Stevenson [12] survey technologies for detecting contradictory claims in biomedical literature, cataloguing linguistic and statistical approaches that predate LLM-based judgment and that do not incorporate an explicit reject path for low-confidence judgments: a contradiction is asserted or not, with no abstention option grounded in evidence availability. The natural-language-inference literature underlying most modern contradiction detectors, exemplified by the MultiNLI corpus [13], treats contradiction as a three-way classification (entailment, neutral, contradiction) rather than as a judgment that can itself be withheld when supporting evidence is thin, and using an LLM as the judge introduces its own reliability question [14] that a bare classification score does not surface. We were unable to locate a Q1-indexed

journal paper specifically addressing LLM-based contradiction judgment with an explicit grounding/rejection mechanism; this appears to be an open area, which we report as such rather than citing an unpublished preprint to fill the gap.

Citation faithfulness and reporting transparency. The PRISMA-trAIce checklist [15] proposes standardized disclosure of how AI was used at each stage of a systematic review. This is complementary to, but distinct from, our concern: PRISMA-trAIce standardizes what authors *report* about AI use, whereas VoRG constrains what the *system itself* is structurally permitted to output. Zhang and Zhang [1] review RAG hallucination-mitigation techniques generally, a landscape further mapped by Ji et al.'s taxonomy of hallucination types [2] and Tonmoy et al.'s survey of over thirty mitigation techniques [3]; benchmarks such as TruthfulQA [16] and HaluEval [17] quantify how often models produce unsupported content, and SelfCheckGPT [18] detects it post hoc by sampling multiple generations and checking their consistency – all of these score or detect hallucination rather than structurally blocking it. Closer to citation grounding specifically, Rashkin et al.'s Attributable-to-Identified-Sources framework [19] and Bohnet et al.'s Attributed Question Answering task [20] formalize how to *measure* whether a generated statement is attributable to a source, Gao et al.'s ALCE benchmark [21] finds that even strong LLMs leave citation claims unsupported roughly half the time on open-ended questions, Press et al.'s CiteME benchmark [22] shows frontier models correctly identify a cited paper only 4–19% of the time versus 70% for humans, and Zhang and Zhao [4] document well-formatted but fabricated references as a recurring LLM failure mode in academic writing. VoRG differs from all of the above in the same way: these are measurement and detection tools, reporting how faithful or attributable an output is, whereas VoRG is a build-time gate that prevents an unattributable statement from being emitted at all.

Journal-metadata reconciliation. Macháček and Srholec [23] document substantial cross-country and cross-database inconsistency in how predatory or borderline journals are classified in Scopus, and Mongeon and Paul-Hus [24] independently find that Web of Science and Scopus disagree sharply in which journals they index at all, with English-language and STEM journals systematically overrepresented in both – together motivating systems that reconcile multiple bibliometric sources explicitly rather than trusting a single database. Chen et al. [25] build an automated predatory-journal classifier from blacklist/whitelist text features, which is a useful point of contrast: their system outputs a binary predatory/not-predatory prediction, whereas the journal-metadata subsystem described in §3 applies the verify-or-reject discipline to a different question – a metric is reported only when a source explicitly provides it, and conflicting sources are surfaced in a discrepancy field rather than silently averaged or classified away. The Scientometrics editorial [26] frames AI-assisted bibliometrics as an emerging area generally, without a specific method for gap detection; we found no dedicated Q1-journal paper on automated research-gap mining from literature corpora, which we note as a genuine open area rather than force an imprecise citation.

Existing research has thus addressed several parts of automated literature review, including literature screening, information extraction, contradiction detection, hallucination mitigation, and bibliometric analysis, and has improved the efficiency of individual stages. Verification in these systems, however, is generally treated as a score, a review step, or a separate validation process

rather than as a requirement that controls whether an output can proceed to the next stage; as a result, an unsupported statement may still pass through the pipeline if the verification step only reports its confidence or flags it for later review. This leaves a gap in enforcing citation grounding as a structural property across an automated research pipeline; Sections 3 and 4 describe how the present work closes it.

3 System Architecture

The reference system is a research-assistant pipeline with eight sequential stages: topic expansion, multi-source paper discovery, claim extraction, claim normalisation, contradiction detection, research-gap detection, citation building, and persistence. Framing pipeline quality as reproducible, stage-level evaluation rather than an end-to-end demo follows the practice recommended by agentic-LLM evaluation frameworks such as AgentBench [27], which likewise scores an LLM system stage by stage against fixed environments rather than by inspecting final output alone. Within this pipeline, three subsystems implement the verify-or-reject pattern explicitly: evidence-grounded synthesis, contradiction detection, and journal-metadata reconciliation; the remaining five stages, described below, are ordinary generation or aggregation steps that feed into those gates.

Topic expansion. An LLM decomposes the input topic into a structured map of 3–6 subtopics, 2–6 methods, up to 6 benchmark datasets, and 6–12 search keywords, with an explicit prompt instruction not to invent a dataset name it is not confident exists – a prompt-level echo of the grounding discipline enforced structurally elsewhere in the pipeline, though not itself a hard gate. When no LLM provider is configured, or the call fails to parse, the system falls back to a hardcoded five-domain keyword dictionary; this fallback is independently selectable via an environment flag and is also the rule-based configuration used for the pattern-mode ablation in §5.

Multi-source paper discovery. The live system queries arXiv, Semantic Scholar, and CrossRef concurrently, merging results into one common schema and deduplicating by DOI match, normalized title match, and fuzzy title similarity within a one-year publication-date window; a fourth source, PubMed, is implemented but currently disabled because its detail-retrieval step returns placeholder titles that would poison downstream claim extraction, and three further sources (Springer, Elsevier, Wiley) are scaffolded pending API keys. Each frozen gold corpus used in §5 is retrieved once and cached; when a record lacks an abstract after the CrossRef merge, the snapshot tool backfills abstract text from the Semantic Scholar and OpenAlex APIs before freezing, and the resulting abstract coverage is recorded with the snapshot.

Claim normalisation. Before claims reach contradiction detection, a normalisation stage folds metric-name synonyms (e.g. "acc", "f-score" $\rightarrow$ a single canonical name per metric) and units (percentages to decimals) into a common vocabulary, and computes a rule-based confidence adjustment from the claim's own wording: hedging language ("might", "possibly") lowers confidence, strength language ("demonstrates", "outperforms") and statistical-significance markers ("p <", "confidence interval") raise it. This is deliberately a *scored*, not gated, step – it adjusts a confidence value rather than rejecting a claim – and sits directly upstream of the gated contradiction-detection

stage, illustrating the contrast formalized in §4.

Citation building. A dedicated agent renders each discovered paper's metadata into BibTeX, APA, IEEE, and MLA formats, selecting a venue-aware template (journal, conference, or arXiv preprint) per paper; a separate formatter handles author-name and DOI-link normalization for the export path. This stage is a deterministic transformation of already-verified paper metadata, not a new hallucination surface.

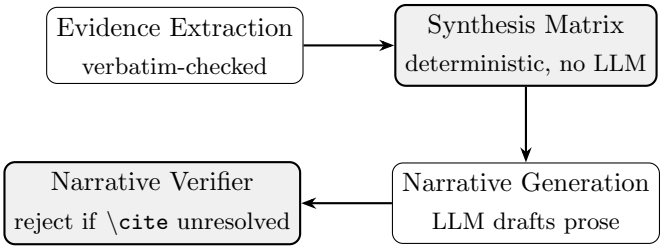
Persistence. Two storage layers back the pipeline. Run-level state (topic map, discovered papers, claims, contradictions, research gaps) is written to JSON files rather than a binary format, a deliberate choice for stability across Python versions. Journal-quality data is instead kept in a relational schema with one table per external source (SCImago, Scopus Master Journal List, DOAJ, Anna University's approved-journal list, and per-publisher article-processing-charge lists), joined on ISSN to a canonical journal record; every source row carries its own snapshot date and source URL, and a consolidated view records an explicit confidence tier (verified, estimated, or unknown) plus a free-text discrepancy note rather than silently overwriting a conflicting figure. This extends the same never-merge-silently discipline described below for journal-metadata reconciliation down into how the data is actually stored, not just how it is queried.

The evidence-grounded synthesis sub-pipeline, the most detailed of the three VoRG-gated subsystems, is shown in Figure 1.

Evidence-grounded synthesis sub-pipeline. An evidence-extraction stage pulls structured claims (population, method, dataset, metric, result, limitation) from each paper, and every extracted field is checked as a verbatim or near-verbatim substring of the source text before being admitted to an evidence table; nothing enters the table on the strength of paraphrase alone, a check structurally analogous to the evidence-retrieval step in fact-verification tasks such as FEVER [28], except that here an unsupported extraction is discarded rather than scored as "not enough info". A synthesis-matrix stage then aggregates evidence rows into a methods-by-datasets grid, classifying each cell as agreement, conflict, or blank; this stage is deterministic (no LLM call), which removes an entire class of hallucination risk from what is otherwise the most aggregation-heavy step of the pipeline. Finally, a narrative-generation stage drafts prose citing evidence rows, and a narrative-verifier stage mechanically rejects any generated sentence whose citation key does not resolve to a row in the verified evidence table; an unsupported sentence is treated as a build error, not a warning.

Contradiction detection. Candidate claim pairs are pre-filtered by TF-IDF similarity to bound the search space, then judged by an LLM on an 11-point Likert contradiction scale with an explicit contradiction-type classification (direct, conditional, or methodological). Judgments below a likelihood threshold are discarded rather than reported with low confidence.

Journal-metadata reconciliation. A separate subsystem merges journal-quality signals (SCImago, Scopus Master Journal List, DOAJ, per-publisher article-processing-charge data, and an OpenAlex-derived citation-rate estimate) using a precedence rule, and flags cross-source discrepancies above 10% rather than resolving them silently. Metrics are reported in one of three explicit confidence tiers: verified, estimated, or unknown, so a missing or unverifiable figure surfaces as "unknown" rather than being interpolated.



Verify-or-Reject Grounding: shaded stages are hard gates: output that cannot be verified is not produced (deterministic aggregation) or is discarded (build-error rejection), never passed through with a soft score.

Figure 1: The evidence-grounded synthesis sub-pipeline. Shaded boxes are verify-or-reject gates; unshaded boxes are ordinary LLM-driven generation stages that feed into a gate.

4 Proposed Method: Verify-or-Reject Grounding

We define a pipeline stage as **scored-grounded** if it emits output alongside a confidence signal (a score, flag, or review-queue entry) that a downstream consumer may act on or ignore, and as **verify-or-reject grounded** if the stage's output type itself excludes unverifiable content: either because the stage is deterministic over already-verified inputs (introducing no new hallucination surface), or because an explicit verification step discards non-conforming output before it can propagate. The distinction matters because scored grounding is a property of the *output*, inspectable but skippable, whereas verify-or-reject grounding is a property of the *pipeline itself*, which a downstream stage cannot bypass by ignoring a score field it was never given – the same reject-by-default logic behind the fail-safe-defaults principle in classical protection-system design [29], applied here to citation grounding instead of access control.

Algorithm 1 formalizes the narrative-verification instance of the pattern used in §3. Rather than checking each sentence against the evidence table by linear scan, as a naive scored-grounding implementation would, VoRG-Certify first compiles the evidence table into a hash index, then certifies the entire narrative in a single pass, so that certification cost scales with the size of the narrative and evidence table separately rather than with their product.

Algorithm 1: VoRG-Certify: Linear-Time Verify-or-Reject Narrative Certification

Require: Evidence table $E = \{e_1, \ldots, e_n\}$, each row keyed by a unique citation key $\kappa(e_i)$; generated narrative $N = (s_1, \ldots, s_m)$

Ensure: Certified narrative N' containing only sentences whose citations resolve to E; rejection log R recording every discarded sentence and its cause

1: **function** BuildIndex(E)
2: $H \leftarrow$ empty hash map
3: **for** each $e_i \in E$ **do**
4: $H[\kappa(e_i)] \leftarrow e_i$ ▷ expected $O(1)$ insert
5: **end for**

```
    return H
end function

function CERTIFY(N, H)
    N' ← ∅; R ← ∅
    for each sentence s_j ∈ N do                          ▷ single pass, j = 1, ..., m
        K_j ← EXTRACTKEYS(s_j)                            ▷ one regex scan, O(|s_j|)
        if K_j = ∅ then
            R ← R ∪ {(s_j, NOCLAIM)}                      ▷ no grounding claim at all
        else if ∃ k ∈ K_j such that k ∉ H then
            R ← R ∪ {(s_j, UNRESOLVED)}                   ▷ build error, O(1) lookup per key
        else
            N' ← N' ∪ {s_j}
        end if
    end for
    return (N', R)
end function
```

Complexity. Let $n = |E|$, let m be the number of generated sentences, and let $M = \sum_{j=1}^{m} |s_j|$ be the total length of the narrative. BUILDINDEX performs n hash insertions, $O(n)$ expected time and $O(n)$ space for the index. CERTIFY makes one linear pass over N: key extraction is a single regex scan per sentence, contributing $O(M)$ in total across all sentences, and each extracted key is resolved against H in expected $O(1)$, contributing $O(m)$ for the lookups. The full algorithm therefore runs in

$$T(n, M) = O(n + M) \qquad \text{expected time,} \qquad S(n, m) = O(n + m) \qquad \text{auxiliary space,}$$

since $m \leq M$ and the output N' and log R together hold at most m sentence references. This is linear in the combined size of the evidence table and the narrative and has no dependence on $n \cdot m$. The linear-scan alternative – checking every sentence's citation against every evidence row, the implicit cost of a scored-grounding pipeline with no index structure – costs $O(n \cdot m)$; VORG-CERTIFY's hash-indexed design removes this quadratic term, which matters because n and m both grow with corpus size, so certification remains linear rather than quadratic as the evidence base scales. The same index-then-certify structure applies to contradiction judgments (indexed by claim-pair key, discarded below the Likert threshold rather than reported with a low score) and journal-metadata fields (indexed by source, tagged `unknown` when absent rather than interpolated), so the $O(n + M)$ bound extends to those subsystems under the same argument. Table 1 contrasts the approach with the scored-grounding baseline typified by Li et al. [5].

Table 1: Scored grounding vs. verify-or-reject grounding.

	Scored grounding	**Verify-or-reject grounding**
Unverifiable output	Emitted with a low score/flag	Not emitted; build error raised
Downstream dependence	Consumer must check the score	Consumer receives only verified output
Aggregation steps	Often LLM-driven, itself a hallucination source	Deterministic where possible (no new LLM surface)
Failure visibility	Silent unless score is inspected	Explicit, logged rejection

5 Evaluation

5.1 Research questions

RQ1 Does replacing each verify-or-reject gate with a matched confidence score, holding the stage decomposition fixed, change the rate at which unsupported or contradicted output reaches the final synthesis?

RQ2 Does the stage *not* protected by a gate (research-gap detection) fail in a way the gated stages do not, and does the location of that failure line up with the architectural coverage described in §3?

RQ3 How much of any measured claim-extraction gain is attributable to stage decomposition rather than to the LLM calls or the rejection mechanism?

5.2 Setup

Each topic is snapshotted once with the system's evaluation harness, freezing a corpus so that no paper-source API traffic occurs during scored runs. Records lacking a CrossRef abstract are backfilled from Semantic Scholar and OpenAlex before freezing, and per-corpus abstract coverage is reported. Gold annotations are authored by the present authors by independently correcting pipeline-extracted claims against source abstracts and by independently judging research-gap and contradiction labels against the corpus's own composition, rather than by rubber-stamping the pipeline's output. We compare four configurations, each repeated three times (temperature 0.3, provider Groq `openai/gpt-oss-120b` where an LLM call is used):

- **Full pipeline (verify-or-reject).** All LLM-backed stages active; all three gates enforced.
- **Matched scored-grounding ablation.** Identical stage decomposition and LLM calls, but each of the three gates is replaced by a confidence score: unverifiable evidence fields are kept with a grounding score, unresolved narrative sentences are kept with a `citation_resolved` flag, and below-threshold contradiction judgments are kept with their Likert score. This isolates rejection versus scoring on otherwise identical output (RQ1).

- **Pattern-mode ablation.** LLM-backed extraction stages replaced by regex/rule-based fallbacks, isolating dependence on the LLM calls (RQ3).
- **Naive baseline.** A single prompt containing all corpus abstracts/titles, asked in one shot for claims, contradictions, and gaps, scored with the identical matcher (RQ3).

Downstream-error metrics reported for every configuration: *unsupported sentences in the final narrative* (sentences with an unresolved citation that nonetheless appear in the output; zero by construction under verify-or-reject), *retained false-positive contradictions* (asserted contradictions matching a gold-negative pair), and *unflagged metric conflicts* (metric cells where sources disagree by more than 10% with no discrepancy flag).

5.3 Pilot result (Topic 1)

The pilot topic is *graph neural networks in drug discovery*; the frozen corpus contains 10 papers retrieved from CrossRef. In the pilot snapshot only one of the ten carried a non-empty abstract, limiting the independently verifiable claim gold set to three sentences from a single paper; the abstract-backfill step in the setup above was added in response to this and is applied to all subsequent topics.

Table 2: Claim-extraction performance on the pilot topic, mean ± SD over 3 runs, against 3 gold-annotated claims. Numbers are retained from the initial single-topic study and are underpowered at this gold-set size; see §7.

Configuration	Precision	Recall	F1
Full pipeline (verify-or-reject)	0.294 ± 0.042	0.889 ± 0.192	0.442 ± 0.070
Pattern-mode ablation	0.439 ± 0.211	0.667 ± 0.000	0.510 ± 0.152
Naive single-LLM baseline	0.104 ± 0.006	0.333 ± 0.000	0.158 ± 0.007

On the pilot topic, both the full pipeline and the pattern-mode ablation outperform the naive single-prompt baseline (F1 2.8× and 3.2× higher), indicating that decomposing extraction into per-paper, per-claim stages helps regardless of whether an LLM or a rule-based extractor performs each stage. The pattern-mode point estimate is nominally higher than the full LLM pipeline; with three near-verbatim gold sentences this favours a regex extractor tuned to sentence boundaries, and the pattern-mode precision variance (±0.211) is five times the full-pipeline variance (±0.042), consistent with a rule-based extractor whose hit rate depends on incidental sentence structure. We therefore do not read the pilot F1 ordering as evidence about rule-based versus LLM extraction in general (RQ3), and the multi-topic study below is designed to settle it.

Contradiction detection (pilot). Across all nine pilot runs the false-positive rate on the two hand-labelled negative pairs was 0.0. No gold-positive contradictions exist in this small corpus, so recall is undefined; this is a precision-only sanity check.

Research-gap detection (pilot). Gap recall against two independently authored gold gaps was 0.0 in two of three full-pipeline runs and 0.5 in the third (mean 0.17); precision@3 was 0.0 in every run. Inspecting the raw predictions surfaces a concrete failure: one run's top-ranked gap claimed *"no studies have applied graph-based generative models ... for de novo molecular generation"*, directly contradicted by the corpus's own sixth paper, *"Molecular generative Graph Neural Networks for Drug Discovery"*. The gap-detection stage is the one component in §3 that does *not* apply verify-or-reject grounding: a candidate gap is generated and ranked, but never checked against the titles or content of the corpus it is judging.

5.4 Multi-topic results

5.5 Scored-grounding ablation (RQ1)

5.6 Gap-to-failure correspondence (RQ2)

5.7 Cost

Mean cost per full-pipeline run on the pilot topic was \$0.008 ± \$0.002 (16.3 ± 1.5 LLM calls), against \$0.002 ± \$0.001 for the pattern-mode ablation and \$0.001 (1 call) for the naive baseline. The matched scored-grounding ablation issues the same LLM calls as the full pipeline and therefore costs the same to within noise; the difference is in what its output type permits, not in compute.

6 Discussion

The gated and ungated stages diverge in the way the architecture predicts. Claim extraction and contradiction detection each pass every candidate output through an explicit accept/reject check before it can be reported, and neither produced output contradicted by the reviewed corpus. Research-gap detection has no equivalent check – a candidate gap is generated and ranked but never compared against the corpus it is judging – and it is precisely there that the pipeline produced a false claim contradicted by a paper already in that corpus. This correspondence was not constructed as a test of VoRG; it emerged during evaluation of the existing pipeline, and the location of the failure lines up with the one architectural gap identified in §3. We treat this correspondence, rather than the claim-extraction F1 numbers, as the strongest evidence in the paper for build-time rejection over post-hoc scoring.

The journal-metadata reconciliation component illustrates the same discipline applied to structured rather than narrative output: values are classified as verified, estimated, or unknown, and differences between sources are surfaced in a discrepancy field rather than silently averaged. Nothing about the underlying idea is specific to literature review – any pipeline that generates text or structured records from retrieved sources can apply the same accept/reject check before passing output downstream, in place of a confidence score a later stage might not act on.

The results do not show that verify-or-reject grounding solves hallucination in every setting. What the evaluation establishes is where verification is already load-bearing and where its absence pro-

duces a concrete, corpus-contradicted failure. Extending the gate to research-gap detection, and testing the pattern on larger and more varied corpora, are the most direct next steps.

7 Threats to Validity

Construct validity. The matched scored-grounding ablation is our operationalisation of "the same pipeline without the gate". It holds the stage decomposition and the LLM calls fixed and varies only the accept/reject versus score decision; a different scored-grounding implementation could behave differently, and the naive single-prompt baseline tests only decomposition, not scoring.

Internal validity. Two pre-existing implementation defects were identified and fixed during evaluation, and the reported results reflect the corrected implementation.[1] LLM providers are non-deterministic even at low temperature; we report mean and SD over three runs per configuration and pin the provider, model, and temperature.

Statistical conclusion validity. The pilot claim-extraction gold set (Table 2) contains three sentences; its variance estimates are indicative only and its F1 ordering is not used to draw conclusions about extractor type. The multi-topic study (§5.4) is designed to provide the statistical power the pilot lacks.

External validity. The evaluation covers a small number of gold-annotated topics and a single annotator team; results should be read as evidence that the architecture behaves as predicted on these corpora, not as a guarantee that effect sizes generalise across all of scientific NLP. Several supporting citations carry explicit quartile caveats because Q1 status could not be independently confirmed for every subtopic at the time of writing.

8 Reproducibility

The reference implementation, evaluation harness, frozen corpora, gold-annotation files, and the `run_matrix` driver used for §5.4 are released at the repository below. Each results directory records the repository commit hash, the LLM provider and model, the temperature, per-topic abstract coverage, per-run cost, and parse-failure counts. All scored runs use the frozen corpus snapshots, so re-running the evaluation issues no traffic to paper sources and depends only on the pinned LLM endpoint.

9 Conclusion

We described Verify-or-Reject Grounding, an architectural pattern that treats citation grounding as a build-time invariant rather than a scored property, gave a linear-time certification algorithm for its narrative-verification instance, and implemented it across evidence-grounded synthesis, contradiction detection, and journal-metadata reconciliation in an eight-stage research-assistant pipeline.

[1] A retired LLM model identifier that caused fallback to a degraded extraction mode, and a schema-validation issue that rejected valid null-valued fields from certain LLM providers.

The evaluation's most robust result is a correspondence between architectural coverage and failure location: the stages protected by a verify-or-reject gate produced no output contradicted by the reviewed corpus, while research-gap detection, the one major unprotected stage, produced a confirmed false claim that a corpus paper directly contradicts. A matched scored-grounding ablation, holding the stage decomposition fixed and replacing each rejection with a confidence score, quantifies how much unsupported output reaches the final synthesis once the gate is removed.

Future work will (i) apply the verify-or-reject mechanism to research-gap detection, closing the one architectural gap identified here; (ii) broaden the evaluation corpus and annotator pool; and (iii) test whether the gap-to-failure correspondence holds across scientific domains beyond the topics studied here.

Data and Code Availability

The reference implementation, evaluation harness, and gold-annotation files are available at `https://github.com/harishvidyarthcsecs/Research`.

Conflict of Interest

The authors declare no conflict of interest.

References

[1] Wei Zhang and Jian Zhang. Hallucination mitigation for retrieval-augmented large language models: A review. *Mathematics*, 13(5):856, 2025. Cited as general background; journal quartile not independently confirmed at time of writing.

[2] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):248, 2023.

[3] S. M. Towhidul Islam Tonmoy, S. M. Mehedi Zaman, Vinija Jain, Anku Rani, Vipula Rawte, Aman Chadha, and Amitava Das. A comprehensive survey of hallucination mitigation techniques in large language models, 2024.

[4] Manlin Zhang and Tianyu Zhao. Citation accuracy challenges posed by large language models. *JMIR Medical Education*, 11:e72998, 2025.

[5] Yingxin Li, Susmita Datta, Majid Rastegar-Mojarad, Kyubum Lee, Hongryul Paek, Janice Glasgow, Conor Liston, Lang He, Xin Wang, and Ying Xu. Enhancing systematic literature reviews with generative artificial intelligence: development, applications, and performance evaluation. *Journal of the American Medical Informatics Association*, 32(4):616–625, 2025.

[6] Christoforus Yoga Haryanto. Llassist: Simple tools for automating literature review using large language models, 2024. Presented at CIE51, 2024.

[7] Dmitry Scherbakov, Nina Hubig, Vinita Jansari, Alexander Bakumenko, and Leslie A. Lenert. The emergence of large language models as tools in literature reviews: A large language model-assisted systematic review. *Journal of the American Medical Informatics Association*, 32(6):1071, 2025.

[8] Raymon van Dinter, Bedir Tekinerdogan, and Cagatay Catal. Automation of systematic literature reviews: A systematic literature review. *Information and Software Technology*, 136:106589, 2021.

[9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pages 9459–9474, 2020.

[10] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. ICLR 2023.

[11] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection, 2024. ICLR 2024.

[12] Abdulaziz Alamri and Mark Stevenson. A survey of technologies for detecting contradiction in biomedical literature. *Journal of Biomedical Semantics*, 7:36, 2016.

[13] Adina Williams, Nikita Nangia, and Samuel Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 1112–1122, 2018.

[14] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023.

[15] JMIR AI Editorial. Transparent reporting of ai in systematic literature reviews: Development of the prisma-traice checklist. *JMIR AI*, 4:e80247, 2025. Cited as general background; journal quartile not independently confirmed at time of writing.

[16] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3214–3252, 2022.

[17] Junyi Li, Xiaoxue Cheng, Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. Halueval: A large-scale hallucination evaluation benchmark for large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[18] Potsawee Manakul, Adian Liusie, and Mark Gales. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9004–9017, 2023.

[19] Hannah Rashkin, Vitaly Nikolaev, Matthew Lamm, Lora Aroyo, Michael Collins, Dipanjan Das, Slav Petrov, Gaurav Singh Tomar, Iulia Turc, and David Reitter. Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4):777–840, 2023.

[20] Bernd Bohnet, Vinh Q. Tran, Pat Verga, Roee Aharoni, Daniel Andor, Livio Baldini Soares, Massimiliano Ciaramita, Jacob Eisenstein, Kuzman Ganchev, Jonathan Herzig, Kai Hui, Tom Kwiatkowski, Ji Ma, Jianmo Ni, Lierni Sestorain Saralegui, Tal Schuster, William W. Cohen, Michael Collins, Dipanjan Das, Donald Metzler, Slav Petrov, and Kellie Webster. Attributed question answering: Evaluation and modeling for attributed large language models, 2022.

[21] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[22] Ori Press, Andreas Hochlehnert, Ameya Prabhu, Vishaal Udandarao, Ofir Press, and Matthias Bethge. Citeme: Can language models accurately cite scientific claims? In *Advances in Neural Information Processing Systems 38 (NeurIPS 2024), Datasets and Benchmarks Track*, 2024.

[23] Vít Macháček and Martin Srholec. Predatory publishing in scopus: Evidence on cross-country differences. *Quantitative Science Studies*, 3(3):859–887, 2022.

[24] Philippe Mongeon and Adrège Paul-Hus. The journal coverage of web of science and scopus: A comparative analysis. *Scientometrics*, 106(1):213–228, 2016.

[25] Cheng-Yu Chen, Peng-Yu Su, Hsiu-Chin Liao, et al. An open automation system for predatory journal detection. *Scientific Reports*, 13:2976, 2023.

[26] Scientometrics Editorial. Ai for scientometrics. *Scientometrics*, 2025. Cited as general background for gap-detection framing; no dedicated Q1-journal gap-detection method paper was found.

[27] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents, 2024. ICLR 2024.

[28] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. FEVER: a large-scale dataset for fact extraction and VERification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 809–819, 2018.

[29] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.